

操作系统课程设计报告

基于虚拟磁盘的 MFD/UFD 二级文件系统设计与实现

课程名称	操作系统课程设计
实验题目	实验二 Linux 二级文件系统
项目名称	mini-kv OSFS 二级文件系统
学生姓名	【请填写】
学号	【请填写】
班级	【请填写】
指导教师	【请填写】
完成日期	2026 年 7 月

摘要

本课程设计实现了一个运行在用户态的二级文件系统 OSFS。程序把普通二进制文件划分为固定大小的磁盘块，在镜像中保存超级块、block bitmap、inode bitmap、用户表、inode 表、主文件目录 MFD、用户文件目录 UFD 和文件数据。格式化时预置 root、Alice、Bob 三个用户；用户记录包含 uid、gid、教学用途的密码哈希和 UFD inode，关闭程序后重新打开镜像仍能完成认证并恢复各自目录。

系统提供 LOGIN、DIR、CREATE、DELETE、OPEN、CLOSE、READ、WRITE、CHMOD、STAT、SEEK 和 TELL 命令。每个 fd 保存独立读写位移，分段读取、连续写入、原位覆盖和追加均由数据块区间 I/O 完成。测试覆盖错误密码、未登录阻断、用户目录隔离、跨用户权限拒绝、重开持久性、跨块读写、空间不足保持旧内容和完整命令行演示。OSFS 使用 mini-kv 仓库的构建与测试基础，但不进入 KV、WAL、snapshot 或 TCP/RESP 执行路径。

关键词：操作系统；二级目录；MFD/UFD；虚拟磁盘；inode；文件描述符

1 课题要求与设计取舍

课程资料把实验二定义为“Linux 二级文件系统”：用二进制文件模拟磁盘，保存用户、inode、超级块和文件信息，完成 dir、create、delete、open、close、read、write、login，并实施读写保护。题目前一页还提出多用户和多级目录目标。这里的“二级”按经典课程设计含义处理为 MFD/UFD 两级，而不是只做一个名字叫 root 的平面目录。MFD 记录用户名到 UFD inode 的映射，每个用户的 UFD 再保存本用户文件名到普通 inode 的映射。

mini-kv 原项目已有 WAL 和 snapshot，但这些能力解决的是键值存储恢复，并不能替代文件系统的 inode、目录和权限。OSFS 因而采用独立 `minikv\_osfs` 入口，复用 CMake、CTest 和跨平台构建，不复用 KV 数据模型。这个取舍让磁盘布局可以按字节解释，也避免为了表面复用而把课程要求藏进键值接口。

要求或模块	本项目中的对应实现
二进制磁盘	VirtualDisk 按 block number 读写 osfs-course.img，重开后状态仍在。
文件系统元数据	Superblock、block bitmap、inode bitmap、user table、inode table 均有固定落盘区域。
二级目录	MFD 保存 user -> UFD inode；root、Alice、Bob 分别拥有真实 UFD inode 和目录块。
用户与 login	用户表持久化 uid/gid/教学密码哈希/UFD inode；错误密码不会切换身份。
文件操作	DIR、CREATE、DELETE、OPEN、CLOSE、READ、WRITE、CHMOD、STAT 均从真实 CLI 可执行。
文件描述符	Handle 保存 read/write offset；READ/WRITE 推进位置，SEEK/TELL 可修改和观察。

读写保护	权限按 root、owner、group、other 顺序判断，跨用户 0600 文件访问被拒绝。
------	---

2 总体结构设计

程序按三层组织。CommandProcessor 解析文本命令，保存登录身份和 fd 表；FileSystem 处理用户目录解析、权限、inode、位图和区间 I/O；VirtualDisk 只执行定长块读写。磁盘结构的编解码又拆到 osfs_disk_layout 模块，目录和文件内容分别由 osfs_directory、osfs_file_io 管理。这样扩展 fd 位移时不需要碰用户认证，修改目录时也不会改写块设备接口。

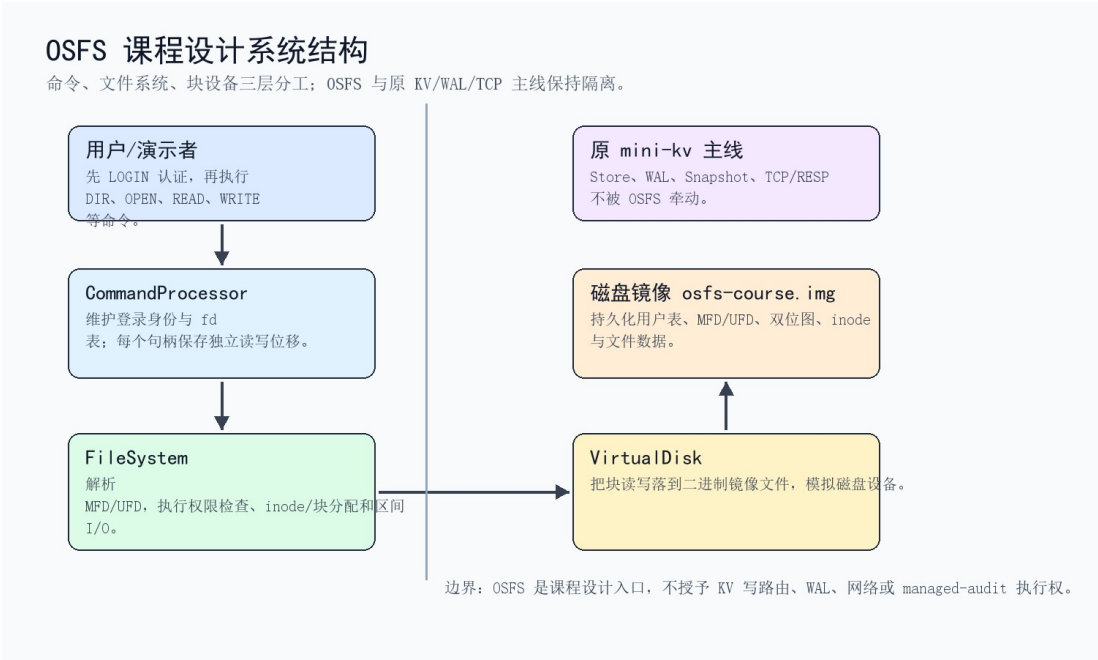


图 1 OSFS 课程设计系统结构

3 磁盘布局与关键数据结构

磁盘格式版本为 2，默认 block size 是 512 字节。Block 0 保存超级块；Block 1 和 Block 2 分别是块位图、inode 位图；Block 3 保存固定长度用户记录；其后是 inode table，最后才是目录与普通文件数据。超级块记录各区域起点、总量、空闲块、空闲 inode、用户数和 MFD inode。打开镜像时先校验 magic、版本和几何信息，v1 镜像会明确要求重新格式化，避免按错误结构解释字节。

每个 inode 固定 96 字节，保存类型、mode、owner uid/gid、size、ctime、atime、mtime 和八个直接块号。目录项固定 64 字节，只保存 used、inode 和 name。用户记录同样为 64 字节，保存用户名、uid、gid、UFD inode 和密码哈希。定长结构配合 static_assert，能够在编译期发现填充造成的格式漂移。

虚拟磁盘布局

二进制文件被切成固定大小块，OSFS 在这些块上组织元数据和文件内容。

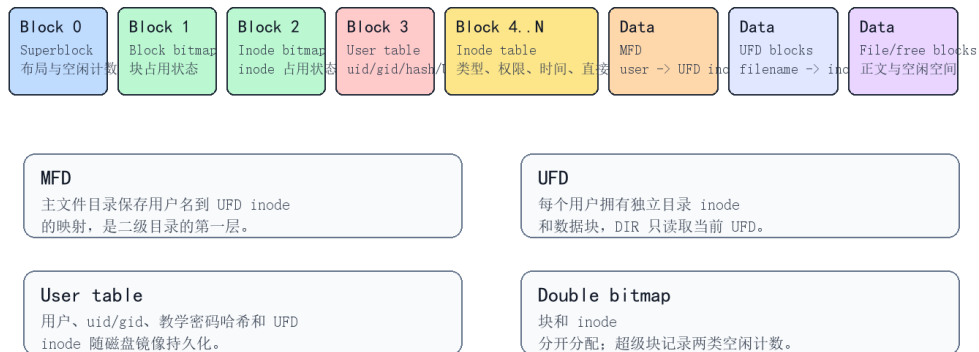


图 2 OSFS 虚拟磁盘布局

4 格式化、认证与二级目录

格式化首先为 MFD 分配 inode 0 和目录块，再为 root、Alice、Bob 各分配一个 UFD inode 与目录块。MFD 的三个目录项指向这三个 UFD，用户表保存相同 UFD inode。登录时，程序从用户表定位记录，使用用户名参与教学哈希计算，再比较磁盘中的 hash。认证失败只返回统一错误；认证成功会切换 uid 并清空旧 fd，防止句柄跨身份遗留。

未带用户名前缀的文件名总是在当前用户 UFD 中解析，所以 Alice 创建的 report 不会进入 Bob 的目录。限定名字 `alice/report` 允许 root 审查，也让测试能够让 Bob 到达目标 inode 后接受权限判断。DIR 默认只列当前 UFD；普通用户不能列别人的 UFD，root 可以使用 `DIR alice`。这两条规则把目录隔离和 inode 权限分开，避免用简单的输出过滤假装二级目录。

5 文件描述符与区间读写

fd 不是文件名的另一个写法。Handle 同时保存文件名、可读/可写标志、read offset 和 write offset。`OPEN w` 先截断；`rw` 保留内容并从位置 0 开始；`a` 把写位置设为 EOF。READ 按实际返回字节数推进读位置，WRITE 只有成功落盘后才推进写位置。SEEK 设置允许方向的偏移，TELL 输出两个位置。

底层区间 I/O 用 `offset / block\_size` 选择直接块，用 `offset % block\_size` 找块内位置。一段数据跨越 512 字节边界时，循环会在当前块末尾切分，再进入下一个块。扩展写入前先检查八个直接块上限和空闲块，不足时不修改数据或 fd 位置；新块先清零，避免间隙暴露旧内容。

文件描述符位移与区间 I/O

fd 保存独立读写位置；底层按块内偏移读取、覆盖或扩展。

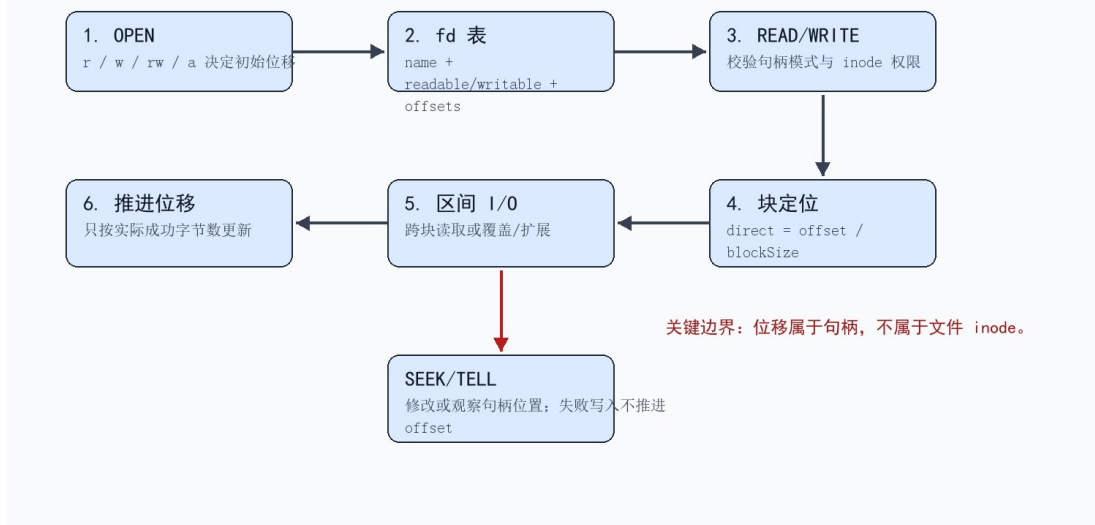


图 3 文件描述符位移与区间 I/O

6 权限模型与失败处理

inode mode 使用八进制权限位。判断顺序为 root、owner、group、other；root uid 0 可以审查全部目录，普通用户只能按自身 uid/gid 和权限位访问。Alice 把文件改成 0600 后，Bob 即使写出 `alice/report` 也会在 inode 权限处得到 permission denied。密码哈希和 mode 都实际参与命令执行，不是 DIR 中的装饰字段。

整文件替换先比较“空闲块 + 当前文件可回收块”和目标需求，空间不足时旧内容保持不变。区间扩展写也在分配前检查所需新增块。创建文件若已取得 inode、随后目录扩容失败，会撤销 inode bitmap；删除则释放全部数据块和 inode，并重算超级块空闲计数。课设磁盘规模较小，重算比依靠多处分散加减更容易审查。

7 关键模块说明

要求或模块	本项目中的对应实现
include/minikv/osfs/virtual_disk.hpp	声明虚拟磁盘接口，提供 create、open、read_block、write_block。
src/osfs_virtual_disk.cpp	负责磁盘镜像大小校验、块边界校验和二进制读写。
src/osfs_disk_layout.*	固定磁盘结构尺寸，读写 superblock、双位图、用户表、inode 和目录块。
src/osfs_filesystem.cpp	格式化、打开镜像、用户表查询和密码认证。
src/osfs_directory.cpp	MFD/UFD 解析、创建、删除、DIR、STAT、CHMOD

	与目录权限。
src/osfs_file_io.cpp	整文件替换、truncate、range read/write、块边界和空间预检。
src/osfs_command_processor.cpp	解析命令，维护登录状态与带 read/write offset 的 fd 表。
src/osfs_main.cpp	提供 minikv_osfs 命令行入口，支持 --disk、--format、--blocks、--script。
tests/osfs_tests.cpp	覆盖认证、二级目录、权限、持久化、offset、跨块 I/O 和失败保持。

8 测试与演示结果

osfs_tests 从 C++ API 检查磁盘格式版本、三个 UFD inode、错误密码、目录隔离、跨用户 0600 拒绝、root 审查、重新打开后的认证与数据持久性。fd 测试连续写 `hello-` 和 `world`，再分段读取、SEEK 覆盖、append 追加；另一个 700 字节文件专门验证 512 字节块边界。空间不足测试确认失败重写不会破坏旧内容。

```

v1633 real MFD/UFD + fd-offset demo

mini-kv osfs course shell
disk=D:\C\mini-kv\cmake-build-v1631\v1633-final-demo.img blocks=96 block_size=512 inodes=64
Commands: HELP, WHOAMI, LOGIN user password, DIR [user], CREATE name, DELETE name, STAT name, CHMOD name mode,
OPEN name r|w|rw|a, READ fd [length], WRITE fd text, SEEK fd offset, TELL fd, CLOSE fd, QUIT
osfs> WHOAMI
not logged in
osfs> CREATE blocked-before-login
ERR login required
osfs> LOGIN alice wrong
ERR authentication failed
osfs> LOGIN alice alice123
OK login alice uid=1000
osfs> WHOAMI
alice uid=1000
osfs> DIR
(empty)
osfs> CREATE report
OK created report
osfs> OPEN report w
OK fd=3
osfs> WRITE 3 course-
OK wrote 7 bytes offset=7
osfs> WRITE 3 design
OK wrote 6 bytes offset=13
osfs> TELL 3
fd=3 read_offset=0 write_offset=13
osfs> CLOSE 3
OK closed 3
osfs> OPEN report r
OK fd=4
osfs> READ 4 6
course
osfs> READ 4 1
-
osfs> READ 4
design
osfs> TELL 4
fd=4 read_offset=13 write_offset=0
osfs> CLOSE 4
OK closed 4
osfs> CHMOD report 0600
OK chmod report 0600
osfs> DIR
name inode physical mode owner size
report 4 20 0600 1000 13
osfs> LOGIN bob bob123
OK login bob uid=1001
osfs> DIR
(empty)
osfs> OPEN alice/report r
ERR permission denied
osfs> CREATE bob-note
OK created bob-note
osfs> DIR
name inode physical mode owner size
bob-note 5 0 0644 1001 0
osfs> LOGIN root root
OK login root uid=0
osfs> DIR alice
name inode physical mode owner size
report 4 20 0600 1000 13
osfs> OPEN alice/report r
OK fd=3
osfs> READ 3
course-design
osfs> QUIT
BYE

```

图 4 OSFS 命令行演示结果

最终脚本从未登录状态开始，先验证命令阻断和错误密码，再以 Alice 创建、连续写、分段读并 chmod 0600。Bob 的 DIR 不出现 Alice 文件，限定访问也被拒绝；root 最后列出 Alice UFD 并读取内容。脚本由真实 minikv_osfs 可执行文件生成 transcript，没有手写成功输出。

```
v1633 full CTest regression

Internal ctest changing into directory: D:/C/mini-kv/cmake-build-v1631
Test project D:/C/mini-kv/cmake-build-v1631
  Start 16: shard_readiness_tests
  Start 13: command_tests
  Start 245: precheck_receipt_split_np_fixture_tests
  Start 240: blocked_rehearsal_split_np_fixture_tests
1/344 Test #240: blocked_rehearsal_split_np_fixture_tests
..... Passed 3.54 sec
  Start 220: sandbox_endpoint_credential_resolver_upstream_echo_non_participation_fixture_tests
2/344 Test #245: precheck_receipt_split_np_fixture_tests
..... Passed 4.03 sec
  Start 250: runtime_approval_gate_archive_np_fixture_tests
3/344 Test #220: sandbox_endpoint_credential_resolver_upstream_echo_non_participation_fixture_tests
..... Passed 3.42 sec
  Start 12: store_tests
4/344 Test #250: runtime_approval_gate_archive_np_fixture_tests
..... Passed 2.95 sec
  Start 235: dfh_echo_split_np_fixture_tests
5/344 Test #13: command_tests
..... Passed 7.03 sec
  Start 230: release_window_readiness_packet_split_non_participation_fixture_tests
6/344 Test #230: release_window_readiness_packet_split_non_participation_fixture_tests
..... Passed 1.89 sec
  Start 215: disabled_precheck_upstream_echo_non_participation_fixture_tests
7/344 Test #235: dfh_echo_split_np_fixture_tests
..... Passed 2.05 sec
  Start 185: production_live_capture_archive_verification_non_participation_fixture_tests
8/344 Test #12: store_tests
..... Passed 3.35 sec
  Start 190: production_live_capture_release_evidence_review_non_participation_fixture_tests
9/344 Test #185: production_live_capture_archive_verification_non_participation_fixture_tests
..... Passed 1.70 sec
  Start 195: catalog_entry_group_split_non_participation_fixture_tests
... output omitted in image; source command completed ...

100% tests passed, 0 tests failed out of 344

Total Test time (real) = 24.05 sec
```

图 5 全量 CTest 验证证据

9 运行与复现步骤

构建只需要 CMake 和 C++20 编译器。课堂演示启动 OSFS 即可，不需要启动 mini-kv TCP 服务。第一次运行使用 `--format` 创建 v2 镜像；后续去掉 `--format` 可以验证用户表、UFD 和文件内容确实持久化。

```
cmake -S . -B build
cmake --build build --target minikv_osfs
./build/minikv_osfs --disk data/osfs-course.img --format --blocks 96 --script 课程设计交付/v1633-osfs-final/osfs-demo-script.txt
```


10 需求核对与对抗性自审

要求或模块	本项目中的对应实现
保存用户、inode、超级块和文件信息	完整：四类信息都位于磁盘镜像固定区域，重开测试通过。
二级目录	完整：MFD 指向 root/Alice/Bob 的真实 UFD inode 与目录块。
dir/create/delete/open/close/read/write	完整：真实 CLI 与 focused CTest 覆盖。
login 用户登录	完整：磁盘用户表与密码校验；错误密码失败且不切换身份。
DIR 展示文件名、物理地址、保护码、长度	完整：同时展示 inode、owner 与 size。
读写保护	完整：chmod 0600 后跨用户读写拒绝。
EXT2 完整组与间接块	部分/教学简化：单 block group、八个直接块，不声称完整 EXT2。

评分老师最可能提出的质疑是：“这是否只是按 owner 过滤一个根目录，并没有真正的二级目录？”回应不靠演示文字：格式化会为 MFD 与三个 UFD 分配不同 inode 和数据块，用户表与 MFD 都保存 UFD inode；Alice 和 Bob 的目录项写入不同 UFD。测试断言三个 UFD inode 不同、双方 DIR 相互隔离，并在重开镜像后再次认证和读取。root 的 `DIR alice` 又能从 MFD/用户表定位 Alice UFD，说明目录关系存在于磁盘结构。

11 设计边界与后续工作

OSFS 不是完整 EXT2。它只实现课程要求的 MFD/UFD 两级，不支持任意深度子目录；inode 只有八个直接块，没有间接块；用户表容量固定，FNV-1a 哈希仅用于教学；系统没有 fsck、崩溃日志和多进程并发。这些限制不会被包装成已完成项。若继续扩展，优先增加 fsck 检查位图、inode 与目录项一致性，再考虑间接块和更安全的密码派生。

12 总结

本次实现把文件拆成了几条可以追踪的关系：用户表把身份连到 UFD，目录项把名字连到 inode，inode 再把权限、长度和数据块连起来。LOGIN 决定从哪个 UFD 开始查找，OPEN 产生带位置的 fd，READ/WRITE 最终落到具体块内偏移。沿着这条路径，课程里的超级块、位图、inode、目录、权限和打开文件表不再是分散概念。

项目最后保留了明确范围：只做 MFD/UFD 二级目录和直接块，不借完整 EXT2 名义夸大成果。OSFS 与原 KV 主线仍然隔离，报告中的每个完成项都能指向源码、测试或真实 transcript。

参考资料

- [1] 操作系统课程设计要求 2026：实验二 Linux 二级文件系统，课程讲义 PDF。
- [2] mini-kv OSFS 源码：include/minikv/osfs、src/osfs_*.cpp。
- [3] mini-kv OSFS 测试与演示：tests/osfs_tests.cpp、tests/osfs_smoke_script.txt。